

EECS 182 Deep Neural Networks
Spring 2025 Anant Sahai

Homework 8

This homework is due on April 13, at 10:59PM.

1. Self-Supervised Linear Purification

Consider a linear encoder — *square* weight matrix $W \in \mathbb{R}^{m \times m}$ — that we want to be a “purification” operation on m –dimensional feature vectors from a particular problem domain. We do this by using self-supervised learning to reconstruct n points of training data $\mathbf{X} \in \mathbb{R}^{m \times n}$ by minimizing the loss:

$$\mathcal{L}_1(W; \mathbf{X}) = \|\mathbf{X} - W\mathbf{X}\|_F^2 \quad (1)$$

While the trivial solution $W = \mathbf{I}$ can minimize the reconstruction loss (1), we will now see how weight-decay (or equivalently in this case, ridge-style regularization) can help us achieve non-trivial purification.

$$\mathcal{L}_2(W; \mathbf{X}, \lambda) = \underbrace{\|\mathbf{X} - W\mathbf{X}\|_F^2}_{\text{Reconstruction Loss}} + \lambda \underbrace{\|W\|_F^2}_{\text{Regularization Loss}} \quad (2)$$

Note above that λ controls the relative weighting of the two losses in the optimization.

(a) Consider the simplified case for $m = 2$ with the following two candidate weight matrices:

$$W^{(\alpha)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad W^{(\beta)} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad (3)$$

The training data matrix $\mathbf{X}$ is also given to you as follows:

$$\mathbf{X} = \begin{bmatrix} -2.17 & 1.98 & 2.41 & -2.03 \\ 0.02 & -0.01 & 0.01 & -0.02 \end{bmatrix} \quad (4)$$

i. **Compute the reconstruction loss and the regularization loss for the two encoders, and fill in the missing entries in the table below.**

Encoder	Reconstruction Loss	Regularization Loss
α	_____	_____
β	0.001	_____

ii. **For what values of the regularization parameter λ is the identity matrix $W^{(\alpha)}$ get higher loss $\mathcal{L}_2$ in (2), as compared to $W^{(\beta)}$?**

(b) Now consider a generic square linear encoder $W \in \mathbb{R}^{m \times m}$ and the regularized objective $\mathcal{L}_2$ reproduced below for your convenience:

$$\mathcal{L}_2(W; \mathbf{X}, \lambda) = \underbrace{\|\mathbf{X} - W\mathbf{X}\|_F^2}_{\text{Reconstruction Loss}} + \lambda \underbrace{\|W\|_F^2}_{\text{Regularization Loss}}$$

Assume $\sigma_1 > \dots > \sigma_m \geq 0$ are the m singular values in $\mathbf{X}$, that the number of training points n is larger than the number of features m , and that $\mathbf{X}$ can be expressed in SVD coordinates as $\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^\top$.

- i. You are given that the optimizing weight matrix for the regularized objective $\mathcal{L}_2$ above takes the following form. **Fill in the empty matrices below.**

$$\widehat{W} = \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix} \cdot \begin{bmatrix} \frac{\sigma_1^2}{\sigma_1^2 + \lambda} & & & & \\ & \frac{\sigma_2^2}{\sigma_2^2 + \lambda} & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & \frac{\sigma_m^2}{\sigma_m^2 + \lambda} \end{bmatrix} \cdot \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix} \quad (5)$$

- ii. **Derive the above expression.**

(Hint: Can you understand $\mathcal{L}_2(W; \mathbf{X}, \lambda)$ as a sum of m completely decoupled ridge-regression problems?)

(Hint: The Frobenius norm is equal to $\|A\|_F^2 := \text{tr}(AA^\top)$, and it is invariant under orthogonal transform. That is, $\|A\|_F^2 = \|UAV^\top\|_F^2$ for any orthogonal matrices U, V , and any rectangular matrix A , as long as U, A, V have compatible shapes.)

- (c) You are given that the data matrix $\mathbf{X} \in \mathbb{R}^{8 \times n}$ has the following singular values:

$$\{\sigma_i\} = \{10, 8, 4, 1, 0.5, 0.36, 0.16, 0.01\}$$

For what set of hyperparameter values λ can we guarantee that the learned purifier $\widehat{W}$ will preserve at least 80% of the feature directions corresponding to the first 3 singular vectors of $\mathbf{X}$, while attenuating components in the remaining directions to at most 50% of their original strength?

(Hint: What are the two critical singular values to focus on?)

2. Hand-Design Transformers

Please follow the instructions in [this notebook](#). You will implement a simple transformer model (with a single attention head) from scratch and then create a hand-designed the attention heads of the transformer model capable of solving a basic problem. Once you finished with the notebook,

- Download `submission_log.json` and submit it to “Homework 8 (Code)” in Gradescope.
- Answer the following questions in your submission of the written assignment:
 - (a) Design a transformer that selects by contents. Compare the variables of your hand-designed Transformer with those of the learned Transformer. **Identify the similarities and differences between the two sets of variables and provide a brief explanation for each difference.**
 - (b) Design a transformer that selects by positions. Compare the variables of your hand-designed Transformer with those of the learned Transformer. **Identify the similarities and differences between the two sets of variables and provide a brief explanation for each difference.**

3. Argmax Attention

Recall from lecture that we can think about attention as being *queryable softmax pooling*. In this problem, we ask you to consider a hypothetical argmax version of attention where it returns exactly the value corresponding to the key that is most similar to the query, where similarity is measured using the traditional inner-product.

- (a) Perform **argmax attention** with the following keys and values:

Keys:

$$\left\{ \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 3 \\ 4 \end{bmatrix}, \begin{bmatrix} 5 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}$$

Corresponding Values:

$$\left\{ \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 4 \\ 3 \end{bmatrix}, \begin{bmatrix} 0 \\ -1 \\ 4 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \right\}$$

using the following query:

$$\mathbf{q} = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$$

What would be the output of the attention layer for this query?

Hint: For example, $\text{argmax}([1, 3, 2]) = [0, 1, 0]$

- (b) Note that instead of using *softmax* we used *argmax* to generate outputs from the attention layer. **How does this design choice affect our ability to usefully train models involving attention?**

(Hint: think about how the gradients flow through the network in the backward pass. Can we learn to improve our queries or keys during the training process?)

4. Justifying Scaled-Dot Product Attention

Suppose $q, k \in \mathbb{R}^d$ are two random vectors with $q, k \sim N(\mu, \sigma^2 I)$, where $\mu \in \mathbb{R}^d$ and $\sigma \in \mathbb{R}^+$. In other words, each component q_i of q is drawn from a normal distribution with mean μ and standard deviation σ , and the same is true for k .

- Define $\mathbb{E}[q^T k]$ in terms of μ, σ and d .
- Considering a practical case where $\mu = 0$ and $\sigma = 1$, define $\text{Var}(q^T k)$ in terms of d .
- Continue to use the setting in part (b), where $\mu = 0$ and $\sigma = 1$. Let s be the scaling factor on the dot product. Suppose we want $\mathbb{E}[\frac{q^T k}{s}]$ to be 0, and $\text{Var}(\frac{q^T k}{s})$ to be $\sigma = 1$. What should s be in terms of d ?

5. Kernelized Linear Attention (Part I)

The softmax attention is widely adopted in transformers, however the $\mathcal{O}(N^2)$ (N stands for the sequence length) complexity in memory and computation often makes it less desirable for processing long document like a book or a passage, where the N could be beyond thousands. There is a large body of the research studying how to resolve this ¹.

Under this context, this question presents a formulation of attention via the lens of the kernel. A large portion of the context is adopted from ². In particular, attention can be seen as applying a kernel over the inputs

¹<https://huggingface.co/blog/long-range-transformers>

²Tsai, Yao-Hung Hubert, et al. "Transformer dissection: a unified understanding of transformer's attention via the lens of kernel" (2019).

with the kernel scores being the similarities between inputs. This formulation sheds light on individual components of the transformer's attention, and helps introduce some alternative attention mechanisms that replaces the “softmax” with linearized kernel functions, thus reducing the $\mathcal{O}(N^2)$ complexity in memory and computation.

We first review the building block in the transformer. Let $x \in \mathbb{R}^{N \times F}$ denote a sequence of N feature vectors of dimensions F . A transformer³ is a function $T : \mathbb{R}^{N \times F} \rightarrow \mathbb{R}^{N \times F}$ defined by the composition of L transformer layers $T_1(\cdot), \dots, T_L(\cdot)$ as follows,

$$T_l(x) = f_l(A_l(x) + x). \quad (6)$$

The function $f_l(\cdot)$ transforms each feature independently of the others and is usually implemented with a small two-layer feedforward network. $A_l(\cdot)$ is the self attention function and is the only part of the transformer that acts across sequences.

We now focus on the self attention module which involves softmax. The self attention function $A_l(\cdot)$ computes, for every position, a weighted average of the feature representations of all other positions with a weight proportional to a similarity score between the representations. Formally, the input sequence x is projected by three matrices $W_Q \in \mathbb{R}^{F \times D}$, $W_K \in \mathbb{R}^{F \times D}$ and $W_V \in \mathbb{R}^{F \times M}$ to corresponding representations Q , K and V . The output for all positions, $A_l(x) = V'$, is computed as follows,

$$\begin{aligned} Q &= xW_Q, K = xW_K, V = xW_V, \\ A_l(x) &= V' = \text{softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V. \end{aligned} \quad (7)$$

Note that in the previous equation, the softmax function is applied rowwise to QK^T . Following common terminology, the Q , K and V are referred to as the “queries”, “keys” and “values” respectively.

Equation 7 implements a specific form of self-attention called softmax attention where the similarity score is the exponential of the dot product between a query and a key. Given that subscripting a matrix with i returns the i -th row as a vector, we can write a generalized attention equation for any similarity function as follows,

$$V'_i = \frac{\sum_{j=1}^N \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^N \text{sim}(Q_i, K_j)}. \quad (8)$$

Equation 8 is equivalent to equation 7 if we substitute the similarity function with $\text{sim}_{\text{softmax}}(q, k) = \exp(\frac{q^T k}{\sqrt{D}})$. This can lead to

$$V'_i = \frac{\sum_{j=1}^N \exp(\frac{Q_i^T K_j}{\sqrt{D}}) V_j}{\sum_{j=1}^N \exp(\frac{Q_i^T K_j}{\sqrt{D}})}. \quad (9)$$

For computing the resulting self-attended feature $A_l(x) = V'$, we need to compute all V'_i $i \in 1, \dots, N$ in equation 9.

- (a) **Identify the conditions that needs to be met by the sim function to ensure that V_i in Equation 8 remains finite (the denominator never reaches zero).**
- (b) The definition of attention in equation 8 is generic and can be used to define several other attention implementations.

³Vaswani, Ashish, et al. "Attention is all you need" (2017).

- (i) One potential attention variant is the “polynomial kernel attention”, where the similarity function as $\text{sim}(q, k)$ is measured by polynomial kernel $\mathcal{K}$ ⁴. **Considering a special case for a “quadratic kernel attention” that the degree of “polynomial kernel attention” is set to be 2, derive the $\text{sim}(q, k)$ for “quadratic kernel attention”. (NOTE: any constant factor is set to be 1.)**
- (ii) One benefit of using kernelized attention is that we can represent a kernel using a feature map $\phi(\cdot)$ ⁵. **Derive the corresponding feature map $\phi(\cdot)$ for the quadratic kernel.**
- (iii) **Considering a general kernel attention, where the kernel can be represented using feature map that $\mathcal{K}(q, k) = (\phi(q)^T \phi(k))$, rewrite kernel attention of equation 8 with feature map $\phi(\cdot)$.**
- (c) We can rewrite the softmax attention in terms of equation 8 as equation 9. **For all the V'_i ($i \in \{1, \dots, N\}$), derive the time complexity (asymptotic computational cost) and space complexity (asymptotic memory requirement) of the above softmax attention in terms of sequence length N , D and M .**
NOTE: for memory requirement, we need to store any intermediate results for backpropagation, including all Q, K, V
- (d) Assume we have a kernel $\mathcal{K}$ as the similarity function and the kernel can be represented with a feature map $\phi(\cdot)$, we can rewrite equation 8 with $\text{sim}(x, y) = \mathcal{K}(x, y) = (\phi(Q_i)^T \phi(K_j))$ in part (b). We can then further simplify it by making use of the associative property of matrix multiplication to

$$V'_i = \frac{\phi(Q_i)^T \sum_{j=1}^N \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^N \phi(K_j)}. \quad (10)$$

Note that the feature map $\phi(\cdot)$ is applied row-wise to the matrices Q and K .

Considering using a linearized polynomial kernel $\phi(x)$ of degree 2, and assume $M \approx D$, derive the computational cost and memory requirement of this kernel attention as in (10).

6. Ridge-Attention

In lecture, you saw how the standard softmax-attention mechanism can be viewed as a softened version of something akin to a nearest-neighbor model in which the value returned for a query reflects a weighted combination of the values that correspond to the keys closest to the query. In this view, the (key, value) pairs in the memory represent a kind of in-context “training data” and the query is a test input for which we want to predict the right output given that data.

- (a) To start, let’s think about why it is possible to efficiently update simple averaging. Let $m = \frac{1}{n} \sum_{i=1}^n x_i$ be the average of n points. **Use m, x_{n+1}, n and simple arithmetic operations to compute $m' = \frac{1}{n+1} \sum_{i=1}^{n+1} x_i$ — the average of all points including the new point x_{n+1} .**
(HINT: Start by multiplying m by n .)
- (b) Let us now shift to thinking about traditional ridge-regression with n training pairs $(\mathbf{x}_i, y_i)$ where $\mathbf{x}_i$

are d -dimensional vectors and y_i are scalars. Let the matrix $A = \begin{bmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix}$ and vector $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ so that

⁴https://en.wikipedia.org/wiki/Polynomial_kernel

⁵https://en.wikipedia.org/wiki/Kernel_method

we can find the familiar closed-form solution

$$\mathbf{w}_* = (A^\top A + \lambda I)^{-1} A^\top \mathbf{y} \quad (11)$$

that allows us to make scalar predictions on any new test input $\mathbf{x}$ by computing $\mathbf{w}_*^\top \mathbf{x}$.

First, **write the two terms $(A^\top A + \lambda I)$ and $A^\top \mathbf{y}$ as sums involving the $\mathbf{x}_i$ and y_i .**

i.e. Complete:

$$(A^\top A + \lambda I) = \lambda I + \sum_{i=1}^n$$

$$A^\top \mathbf{y} = \sum_{i=1}^n$$

- (c) Suppose we wanted to do ridge-self-attention (non-causal – “encoder-style”) with a context length of n and d -dimensional query, key, and value vectors. Recall that this is the style of attention where each of the n queries is applied to the same pool of n (key, value) pairs. The goal is to calculate multi-dimensional ridge-regression predictions, after “training” on the pool of (key, value) pairs, and given the query as a kind of “test” input. (Note: the keys are playing the role of the A matrix in ridge-regression, each query is like the $\mathbf{x}$ we are testing on, and in place of the scalar y_i , we have an entire value vector for a multi-dimensional ridge-regression problem so the $\mathbf{y}$ vector is replaced with a matrix that has a row for each value vector.)

Assume that the cost of inverting a $d \times d$ matrix is $O(d^3)$ and the cost of multiplying two such matrices is also $O(d^3)$. Assume that a $d \times d$ matrix times either a d -dimensional row or column vector costs d^2 operations. You should assume $d < n$.

What is the computational cost of a non-causal ridge self-attention layer?

- ☐ $O(d^4)$
- ☐ $O(nd^2)$
- ☐ $O(n^2 d^3)$
- ☐ $O(n^2 d^2)$
- ☐ $O(n^2)$
- ☐ $O(1)$

(HINT: Do not forget that for a single d -dimensional query vector $\mathbf{q}$, attention needs to return a d -dimensional result.)

- (d) Assume that a ridge self-attention layer is used in a Transformer architecture and there is a downstream loss. **For which of these will backprop successfully pass gradients if we use ridge self-attention?**
- ☐ The ridge λ viewed as a learnable parameter for the self-attention layer.
 - ☐ The keys
 - ☐ The values
 - ☐ The queries
- (e) Now step back. There is a nice trick (called the Sherman–Morrison Formula) by which one can update the inverse of an invertible matrix to which you make a rank-1 update. Let M be an invertible square $d \times d$ matrix and let $\mathbf{u}, \mathbf{v}$ be two d -dimensional vectors. Then:

$$(M + \mathbf{u}\mathbf{v}^\top)^{-1} = M^{-1} - \frac{1}{1 + \mathbf{v}^\top M^{-1} \mathbf{u}} (M^{-1} \mathbf{u})(\mathbf{v}^\top M^{-1}) \quad (12)$$

Assume that a $d \times d$ matrix times either a d -dimensional row or column vector costs d^2 operations, and so does the evaluation of a dyad $\mathbf{u}\mathbf{v}^\top$. Assume that computing a Euclidean inner-product costs d operations. **Assuming that you already had M^{-1} in hand, what is the computational cost of one application of (12)?**

- ☐ $O(d^4)$
 - ☐ $O(d^3)$
 - ☐ $O(d^2)$
 - ☐ $O(d)$
 - ☐ $O(1)$
- (f) Consider implementing causal ridge-self-attention with a context length of n but where the pool of (key, value) vectors that one is querying at position t consists only of the t (key, value) pairs so far. **Describe explicitly how you would compute causal ridge-self-attention in a computationally efficient manner.** Leverage your decomposition of the ridge-regression formula in part (b) of this problem together with the Sherman-Morrison formula from (12) to avoid having to do $O(n^2)$ computations while still calculating causal ridge-self-attention outputs correctly for all n positions in the context. (*HINT: Think recursively. What do you need to track from one time step to the next to avoid repeating work?*)
- (g) Many people consider important the ability to visualize the attention weights. For traditional softmax-attention, these are the outputs of the softmax for a given query vector. They tell you the exact amounts by which the attention outputs at this position are linear combinations of the values being fed in at this and other positions. **For ridge-attention and a given query vector, how would you compute the (possibly negative) weights associated to each of the value vectors in the context?**
- (h) Which of the following position-embedding approaches is easier to *adapt* to ridge-attention (assume the non-causal self-attention variant) without compromising the computational efficiency here:
- ☐ RoPE (*absolute* Rotary Position Embedding)
 - ☐ Learned *relative* position embedding

7. Read a Blog Post: How to train your Resnet

In previous homeworks, we saw how memory and compute constraints on GPUs put limits on the architecture and the hyperparameters (e.g., batch size) we can use to train our models. To train better models, we could scale up by using multiple GPUs, but most distributed training techniques scale sub-linearly and often we simply don't have as many GPU resources at our disposal. This raises a natural question - how can we make model training more efficient on a single GPU?

The blog series [How to train your Resnet](#) explores how to train ResNet models efficiently on a single GPU. It covers a range of topics, including architecture, weight decay, batch normalization, and hyperparameter tuning. In doing so, it provides valuable insights into the training dynamics of neural networks and offers lessons that can be applied in other settings.

Read the blog series and answer the questions below.

- (a) **What is the baseline training time and accuracy the authors started with? What was the final training time and accuracy achieved by the authors?**
- (b) **Comment on what you have learnt.** (≈ 100 words)
- (c) **Which approach taken by the authors interested you the most? Why?** (≈ 100 words)

8. Coding Question: Visualizing Attention

Please run the cells in the [Visualizing_BERT.ipynb](#) notebook, then answer the questions below.

- (a) Attention in GPT: Run part a of the notebook and generate the corresponding visualizations
 - i. **What similarities and differences do you notice in the visualizations between the examples in this part?** Explore the queries, keys, and values to identify any interesting patterns associated with the attention mechanism.
 - ii. **How does attention differ between the different layers of the GPT model? Do you notice that the tokens are attending to different tokens as we go through the layers of the network?**
- (b) BERT pays attention: Run part b of the notebook and generate the corresponding visualizations.
 - i. Look at different layers of the BERT model in the visualizations of part (b) and identify different patterns associated with the attention mechanism. Explore the queries, keys, and values to further inform your answer. **For instance, do you notice that any particular type of tokens are attended to at a given timestep?**
 - ii. **Do you spot any differences between how attention works in GPT vs. BERT? Think about how the model architectures are different.**
 - iii. For the example with syntactically similar but definitionally different sentences, look through the different layers of the two BERT networks associated with sentence a and sentence b, and take a look at the queries, keys, and values associated with the different tokens. **Do you notice any differences in the embeddings learned for the two sentences that are essentially identical in structure but different in meaning?**
 - iv. **For the pre-training related examples, do you notice BERT's bi-directionality in play? Do you think pre-training the BERT helped it learn better representations?**
- (c) BERT has multiple heads!: Run part c of the notebook and generate the corresponding visualizations.
 - i. **Do you notice different features being learned throughout the different attention heads of BERT? Why do you think this might be?**
 - ii. **Can you identify any of the different features that the different attention heads are focusing on?**
- (d) Visualizing untrained attention weights
 - i. **What differences do you notice in the attention patterns between the randomly initialized and trained BERT models?**
 - ii. **What are some words or tokens that you would expect strong attention between? What might you guess about the gradients of this attention head for those words?**
- (e) **Were you able to identify interesting patterns in the visualizations?** If yes, please share some examples (describe in text or paste a screenshot). If not, feel free to use this space for your frustrations.

9. Vision Transformer

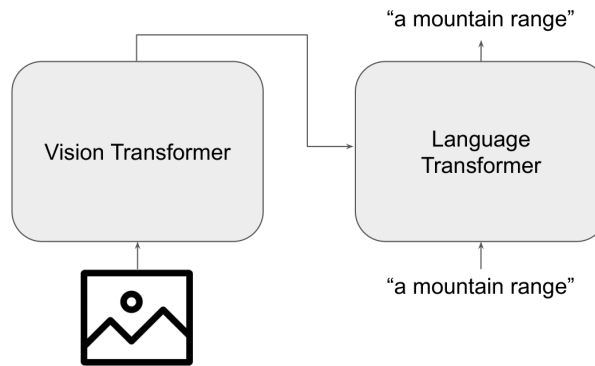


Figure 1: Image captioning model

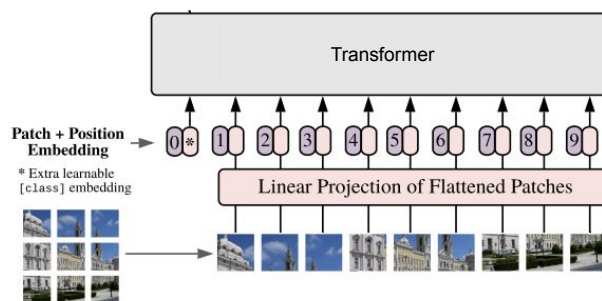


Figure 2: Vision Transformer

You are building a model to perform image captioning. As shown in Figure 1, the model consists of a vision transformer which takes in images and a language transformer which outputs captions. The language transformer will use cross-attention to access the representation of the image.

- (a) For each transformer, state whether it is more appropriate to use a transformer encoder (a transformer with no masking except to handle padding) or decoder (a transformer with autoregressive self-attention masking) and why.

Vision transformer?

- ☐ Encoder-style transformer
☐ Decoder-style transformer

Reason:

Language transformer?

- ☐ Encoder-style transformer
☐ Decoder-style transformer

Reason:

(b) A standard language transformer for captioning problems alternates between layers with cross-attention between visual and language features and layers with self-attention among language features. Let's say we modify the language transformer to have a single layer which performs both attention operations at once. The grid below shows the attention mask for this operation. (For now, assume the vision transformer only outputs 3 image tokens called $\langle \text{ENC1} \rangle$, $\langle \text{ENC2} \rangle$, and $\langle \text{ENC3} \rangle$. $\langle \text{SOS} \rangle$ is the start token, and $\langle \text{PAD} \rangle$ is a padding token.)

(i) One axis on this grid represents sequence embeddings used to make the queries, and the other axis represents sequence embeddings used to make the keys. **Which is which?**

- ☐ Each column creates a query, each row creates a key and a value
- ☐ Each column creates a key and a value, each row creates a query
- ☐ Each column creates a query and a value, each row creates a key
- ☐ Each column creates a key, each row creates a query and a value

(ii) **Mark X in some of the blank cells in the grid to illustrate the attention masks.** (A X marked cell is masked out, a blank cell is not.)

	$\langle \text{SOS} \rangle$	a	mountain	range	$\langle \text{PAD} \rangle$
$\langle \text{SOS} \rangle$					
a					
mountain					
range					
$\langle \text{PAD} \rangle$					
$\langle \text{ENC1} \rangle$					
$\langle \text{ENC2} \rangle$					
$\langle \text{ENC3} \rangle$					

(c) As shown in Figure 2, a vision transformer encodes images by splitting them into patches. Patches get separately encoded into vectors, flattened into a sequence, and then passed into the transformer. What is the Big- O the time complexity of the vision transformer attention operation? Assume you have an image of size $H \times W$ and patches of size $P \times P$. Only consider the time of the attention operation, not the time to produce queries, keys, and values. Queries, keys, and values are all numerical vectors of dimension D .

(d) Some recent papers have reduced the complexity of vision transformer attention by segmenting an image into windows, as shown in Figure 3.

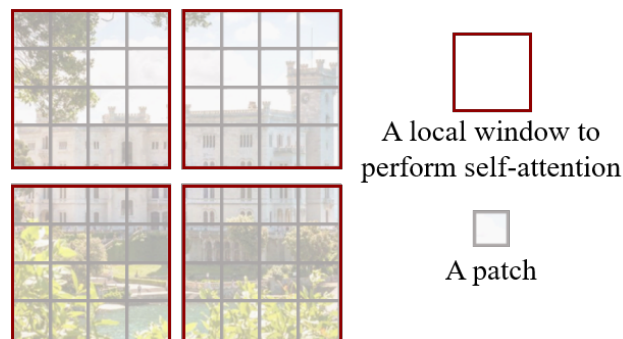


Figure 3: Vision transformer attention with windows

Patches only attend to other patches within the same window. **What is the big- O runtime complexity of the attention operation after this modification?** Assume each window consists of K by K

patches.

10. Homework Process and Study Group

Citing sources and collaborators are an important part of life, including being a student!

We also want to understand what resources you find helpful and how much time homework is taking, so we can change things in the future if possible.

(a) **What sources (if any) did you use as you worked through the homework?**

(b) **If you worked with someone on this homework, who did you work with?**

List names and student ID's. (In case of homework party, you can also just describe the group.)

(c) **Roughly how many total hours did you work on this homework?**

Contributors:

- Dhruv Shah.
- Anant Sahai.
- Olivia Watkins.
- Jake Austin.
- Linyuan Gong.
- Saagar Sanghavi.
- Sheng Shen.
- David M. Chan.
- Shaojie Bai.
- Angelos Katharopoulos.
- Hao Peng.
- Romil Bhardwaj.
- Shivam Singhal.
- Kevin Li.
- Bryan Wu.
- Yuxi Liu.